

Lecture 8: Practice with Threads

Lecture 8 has no slides. It is a list of tasks to practise the concepts from Lecture 7: race conditions, atomics, deadlock, condition variables, the producer-consumer problem and semaphores. Every task has a story, the ready functions, the code and the questions. The code of each task is in its folder on GitHub, for example [Task1](#).

Tasks 6, 8, 9 and 11 start with an introduction to a new tool or problem: condition variables, the producer-consumer problem, semaphores and the dining philosophers.

	task	page
Task 1	The ticket shop	2
Task 2	The discount code	5
Task 3	Statistics in a 5G base station	7
Task 4	Robots in a warehouse	10
Task 5	The pizzeria	13
Task 6	The lobby of an online game	16
Task 7	Threads take turns	19
Task 8	The video decoder	21
Task 9	At most 3 decoders at the same time	25
Task 10	The decoder with two semaphores	28
Task 11	The dining philosophers	31

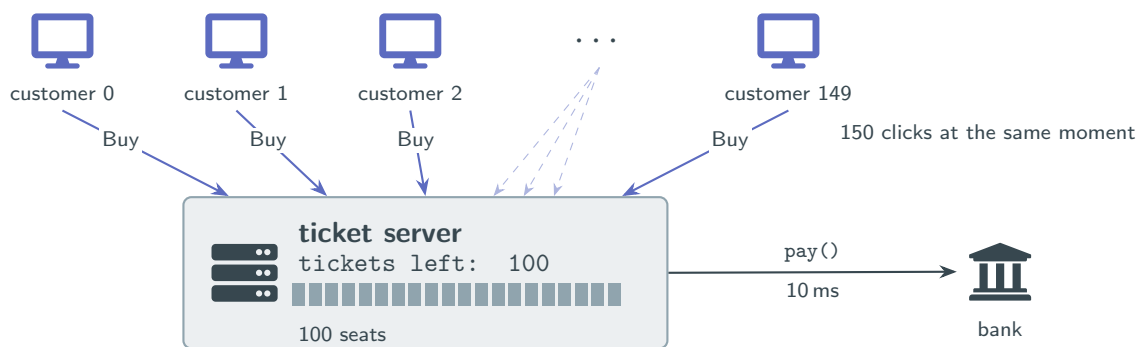
Computer Hardware ■ Lecture 8: Practice with Threads

Task 1: The ticket shop

A company has hired you to check the code of its ticket shop. Look at the code below and judge the dangers. Is there a race condition here? A deadlock? What do you think?

The situation

A concert hall has **100 seats**, and **150 people** want a ticket. Every customer is a separate thread with a unique number from 0 to 149, and all of them click *Buy* at the same time.



150 customers, 100 tickets: who gets one?

Ready code (in `tickets.h`, we do not look inside)

Functions

`pay()` simulates connecting to the bank and the customer paying for the ticket:
10 ms

Constants

`SEATS` 100 seats in the hall
`CUSTOMERS` 150 customers, one thread each

The code

The template is in [Task1](#). This is `tickets.cpp`:

```
int tickets = SEATS;           // tickets left
bool got[CUSTOMERS] = {false}; // did the customer get a ticket?

void customer(int id) {
    if (tickets > 0) {          // 1. check: any tickets left?
        pay();                  // (the customer pays)
        tickets--;              // 2. act: buy
        got[id] = true;
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < CUSTOMERS; id++)
        threads.emplace_back(customer, id);
    for (auto &t : threads)
        t.join();

    std::cout << "tickets left: " << tickets << "\n";
}
```

In the task folder, make `run` compile the program and runs it.

How `main()` starts 150 threads

In Lecture 7 every thread had its own variable: `std::thread a(lunch);`. For 150 threads we keep them in a list instead, a `std::vector`:

```
std::vector<std::thread> threads; // an empty list of threads
for (int id = 0; id < CUSTOMERS; id++)
    threads.emplace_back(customer, id); // start a new thread: customer(id)
for (auto &t : threads)                // for every thread in the list ...
    t.join();                          // ... wait until it ends
```

`threads.emplace_back(customer, id)` creates a new `std::thread` at the end of the list. The thread starts at once and runs `customer(id)`: the first one with `id = 0`, the last one with `id = 149`. It does the same as `std::thread t(customer, id);`, only the thread is kept in the vector, so that we can `join()` all of them later. The other tasks start their threads in the same way.

Your tasks

- 1 Do you see a danger? What exactly can go wrong? Suppose only one ticket is left, and two customers click *Buy* at the same moment. Describe, step by step, how both of them can get a ticket.
- 2 Write a function `check()` that checks whether the shop sold the tickets correctly: no more tickets than seats, and the tickets sold plus the tickets left add up to `SEATS`. It prints `OK` or `ERROR`. Call it at the end of `main()` and run the program several times. Does your check catch the problem every time?

- 3 Fix the danger. Does your check pass now?
- 4 `pay()` talks to the bank, and in real life it can take several seconds. While one customer pays under the mutex, all the others wait. It is possible to move `pay()` **outside** the mutex and keep the shop safe. Think how to do it, write it, and check it with `check()`. Measure how long the program takes before and after: `time ./build/tickets`.
- 5 Would it be enough to change `int tickets` to `std::atomic<int> tickets` and keep the same `if`? Try it.

Task 2: The discount code

An online shop gave a customer the discount code *PROM050*. The code may be used **at most 3 times**: the shop checks that in the code, with a counter. A week later the accountants saw that the code had been used several hundred times, although the limit was 3. How is this possible?

The situation

Every click on *Use code* starts a new thread that calls `redeem()`. The function checks the counter used, verifies the order (slow: stock, price, address) and counts the use.



Ready code (in `coupon.h`, we do not look inside)

Functions

`verify_order()` simulates the shop checking the order (stock, price, address): 100 ms

Constants

`MAX_USES` 3: the code may be used at most 3 times

This bug is not only a wrong number in a report. Here it costs the shop money, and the same mistake in a check of a limit (a number of tries, a budget, a quota) is a **security hole**. Programmers call it *time of check to time of use* (TOCTOU): the program checks a rule at one moment and uses the result a moment later, when the rule may be already broken.

The code

The template is in [Task2](#). This is coupon.cpp:

```

const int CLICKS = 100;           // how many people click "Use code" at the same time

int used = 0;                     // how many times the code was used

// returns true if the discount was given
bool redeem() {
    if (used >= MAX_USES) { // 1. check: is the limit used up?
        return false;
    }
    verify_order();           // (slow)
    used++;                   // 2. act: count this use
    return true;
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < CLICKS; i++)
        threads.emplace_back(redeem);
    for (auto &t : threads)
        t.join();
    std::cout << CLICKS << " clicks, the code was used " << used
                << " times (limit " << MAX_USES << ")\n";
}

```

In the task folder, make run compiles the program and runs it.

Your tasks

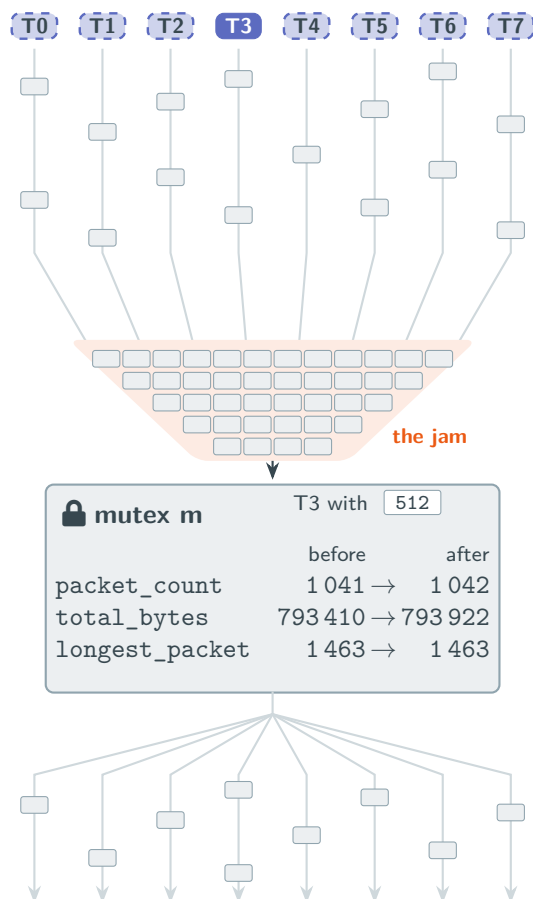
- 1 Does the limit of 3 uses really work? Suppose the code has already been used 2 times, so one use is left. Two customers click *Use code* at the same moment. Describe, step by step, how both of them can get the discount.
- 2 Set CLICKS to 2, 10, 100 and 1000. Run each one 3 times and write the results in a table. How many times was the code used? What do you notice?
- 3 Fix redeem() so that the code works at most 3 times, for any number of clicks.
- 4 verify_order() is slow, and while one click verifies its order under the mutex, all the others wait. It is possible to move verify_order() **outside** the mutex and still allow at most 3 uses. Think how to do it (Task 1, question 4 is similar) and write it.

Task 3: Statistics in a 5G base station

A 5G base station receives millions of packets per second from the phones around it, and eight threads process them. The station keeps three statistics: how many packets have come so far, how many bytes they had together (for monitoring and billing), and the length of the longest packet so far (to size the buffers). The code is correct, but the profiler shows that the threads spend most of their time waiting for one mutex. Make it faster.

Note: this is a very simplified model, made for teaching.

The situation



1 Loose

Eight threads, each with its own packets. Every thread takes a packet and calls `on_packet(len)`. Plenty of room: the threads work at the same time.

2 The jam

All of them want the same mutex. Only one can go in; the other seven wait in front of the door. More threads only make the jam longer.

3 One at a time

Inside the mutex, T3 updates the statistics with its packet of 512 bytes, and gets the number 1 041 back.

4 Loose again

After the mutex every thread goes on alone with its packet: no more waiting.

8 threads, but at the mutex only 1 of them works at a time

The code

The template is in [Task3](#). This is stats.cpp:

```
// To check that your solution is correct, change TEST from 0 to 1.
const int TEST = 0;

#include <stdint>
#include <mutex>
#include "packets.h"    // ready: the speed measurement and the tests

std::mutex m;
uint32_t packet_count = 0; // how many packets so far; the next packet gets this number
uint64_t total_bytes = 0;  // the length of all packets together
uint32_t longest_packet = 0; // the length of the longest packet so far

// called for every packet; returns its sequence number
uint32_t on_packet(uint32_t len) {
    std::lock_guard<std::mutex> lock(m);
    uint32_t number = packet_count++;
    total_bytes += len;
    if (len > longest_packet) {
        longest_packet = len;
    }
    return number;
}

int main() {
    // TEST 0: measure the speed, TEST 1: run the tests
    return run(TEST, total_bytes, longest_packet);
}
```

In the task folder, make run compile the program and runs it.

The program measures the speed: 8 threads send 1 000 000 packets each, and it prints how many million packets per second on_packet() handled.

Your tasks

- 1 Run it. How many packets per second does the version with the mutex handle?
- 2 Replace the mutex with std::atomic. To check your solution, change TEST from 0 to 1 at the top of stats.cpp: the program then does not measure, but runs ready tests. Do all tests pass? If not, why not, when every variable is atomic?
- 3 Fix on_packet() so that all tests pass, and keep it fast. Hint: most packets are not longer than longest_packet, and such a packet does not change it. Does it have to lock anything?
- 4 Set TEST back to 0. How many times faster is it now than in question 1? Why?

The tests (TEST 1)

They are ready in packets.h; you do not write them. 8 threads send packets, and the program checks four things: the sequence numbers are unique, the byte counter is exact, longest_packet never goes down, and longest_packet is right at the end.

Atomic variables

`std::atomic<int>` (from `<atomic>`) is an `int` whose every operation happens in **one step**: no other thread can come in the middle of it. In the same way `std::atomic<uint32_t>` is an atomic `uint32_t`.

```
std::atomic<int> x{5};           // declare: an atomic int that starts at 5

int old = x.fetch_add(3);        // add 3, return the value before: x = 8, old = 5
x++;                             // the same as x.fetch_add(1): x = 9

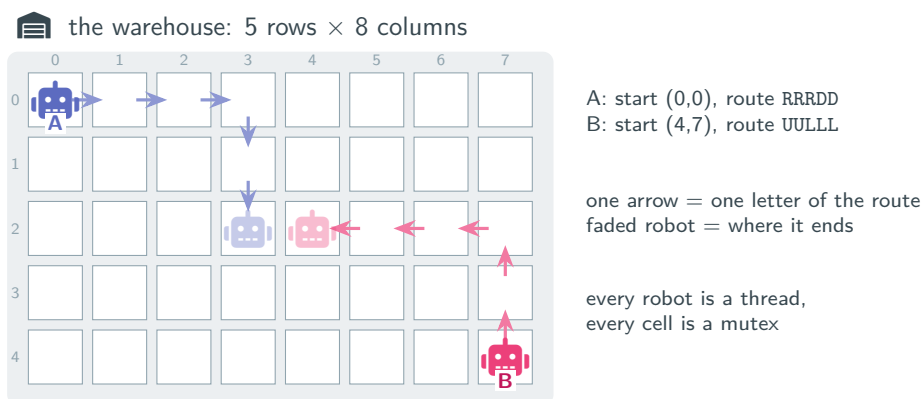
int now = x.load();              // read: now = 9
if (x > 7) { /* ... */ }         // a plain read works too, it is the same as x.load() > 7

x = 10;                          // write: x = 10
```

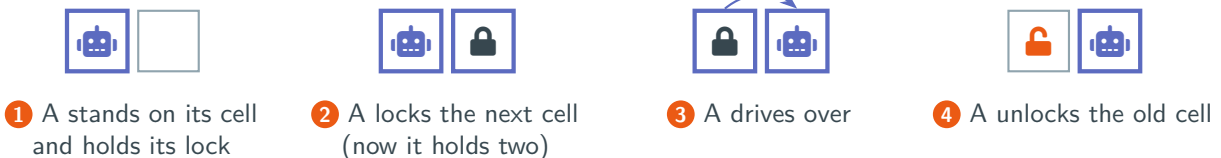
Computer Hardware ■ Lecture 8: Practice with Threads

Task 4: Robots in a warehouse

A company runs an automatic warehouse: robots drive around a grid of cells. The company says that its robots can never get stuck. Prove them wrong: write routes after which the robots are stuck for ever.

The situation

Every robot is a thread, and every cell of the grid is a mutex. A robot **holds** the lock of the cell it stands on. One step takes four moments:



So a robot never stands on no cell, and two robots never stand on one cell.

The program is **ready**: you do not change the code, only the input. The routes are in the file `routes.txt`. The grid has 5 rows (0 to 4) and 8 columns (0 to 7); the cell (0,0) is in the top left corner.

```
# Routes of the robots. The grid has 5 rows (0 to 4) and 8 columns (0 to 7).
# The cell (0,0) is the top left corner.
#
# robot row column route ms per step
# route: R = right, L = left, U = up, D = down

A 0 0 RRRDD 200
B 4 7 UULLL 300
```

In the task folder, `make run` compiles the program and runs it.

A route is a list of steps: R right, L left, U up, D down. The last number is the time of one step of this robot in milliseconds. When a robot has finished its route, it leaves the warehouse: it unlocks its cell and disappears from the grid.

After every step the program prints where the robot went and draws the grid:

```

$ ./build/robots                                # reads routes.txt
start:
  A . . . . .
  . . . . .
  . . . . .
  . . . . .
  . . . . . B

robot A: (0,0) -> (0,1)
  . A . . . . .
  . . . . .
  . . . . .
  . . . . .
  . . . . . B

...
$ ./build/robots other.txt                      # reads other.txt

```

When every robot that has not finished has waited for more than 2 s, the program prints who waits for whom and stops:

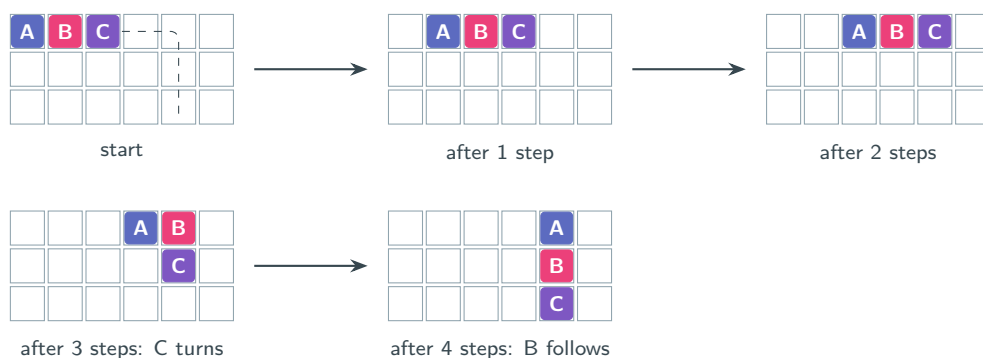
```

DEADLOCK: nobody can move.
  robot A waits for (2,4), which robot B holds
  robot B waits for (2,3), which robot A holds

```

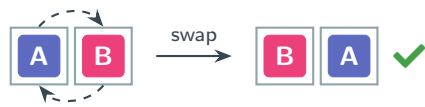
Your tasks

- 1 Write routes for 2 robots after which they are stuck for ever (a deadlock).
- 2 Does your deadlock happen every time, or only sometimes? Change the speeds of the robots and check.
- 3 Three robots drive in a line, like a snake: A right behind B, B right behind C, all on the same path. Each one may have a different speed, so a faster robot catches up with the slower one in front and waits for its cell. Can they get stuck for ever? Why? Write such routes and check.



Each robot drives into the cell that the robot in front of it has just left.

- 4 After your report, the engineers added a safety rule for the case from your report: when two robots want **each other's** cells, they **swap** places. This works only when A wants the cell of B **and** B wants the cell of A. In every other case a robot waits, as before:



A wants the cell of B,
B wants the cell of A: they swap



D wants the cell of A, but A wants
the cell of B: no swap, D waits

The boss announces that the deadlock problem is solved. Is the boss right? If not, write routes after which the robots are stuck for ever even with the safety rule. You may use as many robots as you want. To switch the rule on, add the line `safety on` at the top of your routes file.

Hint

Draw the robots on paper first: for every robot, which cell does it hold, and which cell does it want?

Computer Hardware ■ Lecture 8: Practice with Threads

Task 5: The pizzeria

Three cooks make three kinds of pizza, and sometimes they need the same ingredients. The owner says that something is wrong with the kitchen software: some evenings the kitchen stops, and on other evenings the number of pizzas on the board does not match the till. This program has **two** bugs. Find both, tell them apart and fix them.

This task joins Tasks 1 to 4: one race condition and one deadlock in one program.

The kitchen

Every ingredient is a mutex: only one cook at a time takes from it. A cook takes the ingredients in their own order, holds all of them until the pizza is baked, and then gives them back.



After every pizza the cook adds 1 to the number on the board, pizzas. Every cook makes 10 pizzas, so at the end the board should show 30.

Ready code (in kitchen.h, we do not look inside)

Functions

take(m)	take an ingredient: lock its mutex
bake()	bake the pizza: 2 ms
write_on_board()	write the number on the board: 2 ms
serve(ms)	bring the pizza to the table: A 20 ms, B 30 ms, C 40 ms

Constants

PIZZAS	10: every cook makes 10 pizzas
--------	--------------------------------

The code

The template is in [Task5](#). This is pizzeria.cpp:

```
// every ingredient is a mutex: one cook at a time takes from it
std::mutex tomato, cheese_sauce, mozzarella, cheddar, jalapeno, pepperoni;

int pizzas = 0;           // the number on the board: all pizzas so far

void add_to_board() {      // pizzas++, written out
    int n = pizzas;        // read the number on the board
    write_on_board();      // (takes a moment)
    pizzas = n + 1;        // write the new number
}

void cook_a() {            // Fat Peppa
    for (int i = 0; i < PIZZAS; i++) {
        take(tomato); take(mozzarella); take(pepperoni);
        bake();
        pepperoni.unlock(); mozzarella.unlock(); tomato.unlock();
        add_to_board();
        serve(20);
    }
}

void cook_b() {            // Fat Cheese
    for (int i = 0; i < PIZZAS; i++) {
        take(cheese_sauce); take(mozzarella); take(jalapeno); take(cheddar);
        bake();
        cheddar.unlock(); jalapeno.unlock();
        mozzarella.unlock(); cheese_sauce.unlock();
        add_to_board();
        serve(30);
    }
}

void cook_c() {            // Diavola
    for (int i = 0; i < PIZZAS; i++) {
        take(jalapeno); take(tomato); take(mozzarella); take(pepperoni);
        bake();
        pepperoni.unlock(); mozzarella.unlock(); tomato.unlock(); jalapeno.unlock();
        add_to_board();
        serve(40);
    }
}

int main() {
    std::thread a(cook_a), b(cook_b), c(cook_c);
    a.join(); b.join(); c.join();
    std::cout << "pizzas on the board: " << pizzas
                << " (should be " << 3 * PIZZAS << ")\n";
}
```

In the task folder, make run compiles the program and runs it.

Your tasks

- 1 Run the program 10 times. Write down what happens each time: it hangs, it prints a wrong number, or it prints 30. (A program that hangs: stop it with Ctrl+C.)
- 2 Find the race condition on pizzas. Which lines, and what exactly goes wrong?
- 3 Fix the counter. `std::atomic` or a mutex? Why?
- 4 Draw who waits for whom when the program hangs, and find the circle.
- 5 Only cook B uses the cheese sauce and the cheddar. Are they part of the deadlock?
- 6 Cooks A and C both use pepperoni. Is it part of the deadlock?
- 7 Choose one order for everybody. A pizza is made from the bottom up: sauces first, then cheeses, then toppings: tomato sauce, cheese sauce, mozzarella, cheddar, jalapeño, pepperoni. Change the order of cooks B and C to follow it.
- 8 Run the program 10 times again. Does it ever hang? Is the number always right?

Task 6: The lobby of an online game

Tasks 6 and 7 use a new tool, the **condition variable**. Read this introduction first.

Condition variables: how they work

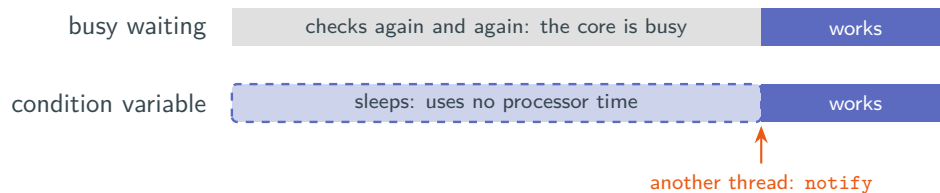
Often a thread has to wait until something happens: until all players are here, until a frame is ready, until it is its turn. With what you know so far, the thread checks the condition in a loop:

```
m.lock();
while (!ready) {           // not yet? look again
    m.unlock();           // let the others change ready
    m.lock();
}
m.unlock();
```

This works, but the thread does nothing useful and still uses a whole core of the processor: it asks again and again, millions of times per second. This is called **busy waiting**.

Condition variable

A **condition variable** (`std::condition_variable`) lets a thread **sleep** until another thread wakes it up. A sleeping thread uses no processor time.



A condition variable always comes with three things:

1. The **state** that the threads wait for (for example a counter of players), protected by a **mutex**.
2. The **waiting** thread. It locks the mutex and checks the condition in a `while` loop. If the condition is false, it calls `wait()`. `wait()` unlocks the mutex and puts the thread to sleep, in one step. When the thread is woken up, `wait()` locks the mutex again and returns.
3. The **waking** thread. It locks the mutex, changes the state, and calls `notify_one()` (wakes up one waiting thread) or `notify_all()` (wakes up all of them).

```

std::mutex m;
std::condition_variable cv;
bool ready = false;

void waiter() {
    std::unique_lock<std::mutex> lock(m);
    while (!ready)           // while, not if: check again after waking up
        cv.wait(lock);      // unlock m and sleep; lock m again when woken up
    // here ready == true and m is locked
}

void waker() {
    std::lock_guard<std::mutex> lock(m);
    ready = true;
    cv.notify_all();         // wake up everybody who waits on cv
}

```

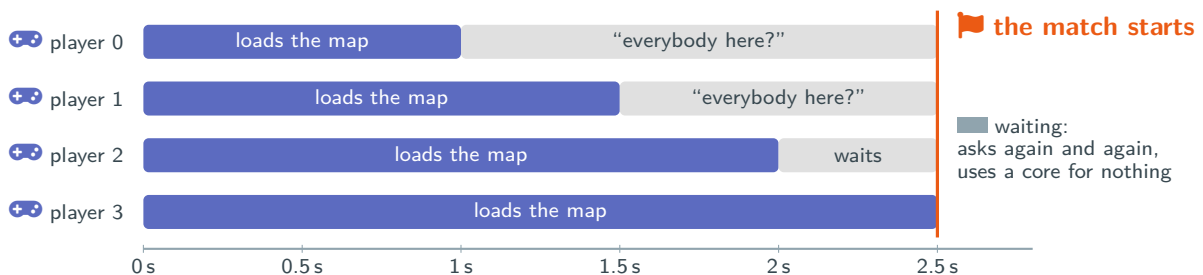
Three details matter:

- **unique_lock, not lock_guard.** `wait()` has to unlock and lock the mutex again. A `lock_guard` cannot do that; a `unique_lock` can.
- **while, not if.** After waking up, the condition may be false again: another thread may have been faster, and a thread can even wake up without any notify (a *spurious wakeup*). So always check again.
- **Check the condition under the mutex.** If the waiter checked without the mutex, the waker could change the state and call `notify` between the check and `wait()`. Nobody would be waiting yet, the notify would be lost, and the waiter would sleep for ever (a *lost wakeup*).

A mutex protects the data. A condition variable lets a thread sleep until the data changes.

The task

A match starts when all 4 players have loaded the map. Every player is a thread that loads the map (it takes a different time for every player) and then waits for the others. Now the players ask again and again: “is everybody here?”, and the game server uses its whole processor just for waiting.



Ready code (in `lobby.h`, we do not look inside)

Functions

load_map(id)	the player loads the map: player 0 takes 1s, player 1 takes 1.5s, ..., player 3 takes 2.5s
start_match(id)	the match starts for this player (prints a line)
Constants	
PLAYERS	4 players in a match

The template is in [Task6](#). This is lobby.cpp:

```

int loaded = 0;           // how many players have loaded the map
std::mutex m;

void player(int id) {
    load_map(id);
    m.lock();
    loaded++;
    while (loaded < PLAYERS) {    // not everybody? ask again
        m.unlock();
        m.lock();
    }
    m.unlock();
    start_match(id);
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < PLAYERS; id++)
        threads.emplace_back(player, id);
    for (auto &t : threads)
        t.join();
}

```

In the task folder, make run compiles the program and runs it.

Your tasks

- 1 Run it with `time ./build/lobby` (Linux, macOS). How much processor time (user and sys) did the players use, when they mostly waited?
- 2 Replace busy waiting with a `std::condition_variable`. Who should wake the players up? Run it with `time` again.
- 3 `notify_one()` or `notify_all()`? Why?
- 4 A match has 3 rounds, and before each round we wait until everybody is ready. Is it enough to set `loaded` back to 0 when a round starts? Write it and run it. What happens, and why? How can you fix it?

`time`

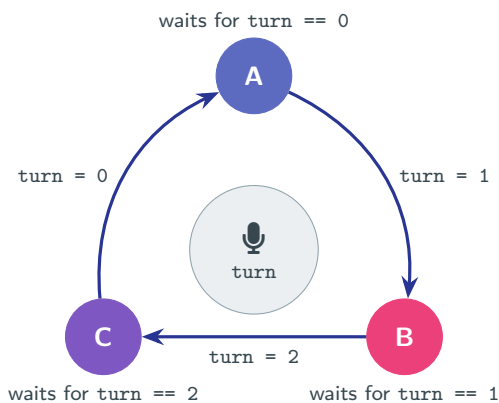
real is the time on the clock. user is the processor time spent in your program, and sys the processor time spent in the operating system for your program. On Windows, compare the processor usage in the Task Manager instead.

Computer Hardware ■ Lecture 8: Practice with Threads

Task 7: Threads take turns

Three threads must print in turn: A B C A B C ... Now each of them asks again and again whether it is its turn. Make them sleep instead.

This task uses condition variables: read the introduction in Task 6 first. In Task 6 every player waited for the **same** thing (all players are here). Here every thread waits for something **different**: thread A waits for `turn == 0`, thread B for `turn == 1`, thread C for `turn == 2`.



the output, one letter every 300 ms:

A B C A B C A ...

While one thread speaks, the other two ask again and again: "is it my turn?"

Ready code (in `turns.h`, we do not look inside)

Functions

`say(name)` print the letter: 300 ms

Constants

`ROUNDS` 10: every thread says its letter 10 times

The code

The template is in [Task7](#). This is `turns.cpp`:

```

const int THREADS = 3;

int turn = 0;           // whose turn it is: 0 = A, 1 = B, 2 = C
std::mutex m;

void player(int id) {
    char name = 'A' + id;
    for (int i = 0; i < ROUNDS; i++) {
        m.lock();
        while (turn != id) {    // not my turn? ask again
            m.unlock();
            m.lock();
        }
        m.unlock();
        say(name);              // takes 300 ms: not under the mutex
        m.lock();
        turn = (turn + 1) % THREADS;
        m.unlock();
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < THREADS; id++)
        threads.emplace_back(player, id);
    for (auto &t : threads)
        t.join();
    printf("\n");
}

```

In the task folder, make `run` compile the program and runs it.

Your tasks

- 1 Run it with `time ./build/turns`. How much processor time (user) did the threads use, when they mostly waited for their turn?
- 2 Replace busy waiting with a `std::condition_variable`. Run it with `time` again.
- 3 Try `notify_one()`, then `notify_all()`. Run each version several times. Which one always works? Why?
- 4 With 2 threads (set `THREADS` to 2), `notify_one()` works every time. With 3 the program sometimes hangs. Why?

Hint for question 4

`notify_one()` wakes up **one** of the waiting threads, but you do not choose which one.

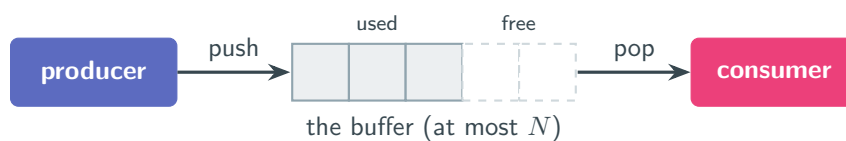
Task 8: The video decoder

Task 8 is about a famous problem of concurrency. Read this introduction first.

The producer-consumer problem

The **producer-consumer problem**, also called the **bounded buffer problem**, is one of the classic problems of concurrent programming. Edsger Dijkstra described it in 1965.

Some threads **produce** things (the **producers**), other threads **use** them (the **consumers**). They work at different speeds, so between them there is a **buffer**: a queue that holds at most N things.



There are two kinds of waiting:

- **The buffer is full:** the producer waits until the consumer takes something.
- **The buffer is empty:** the consumer waits until the producer adds something.

With mutexes only, the program is correct: the mutex protects the queue, so nothing is lost or broken. But a mutex cannot put a thread to sleep until something changes. A thread that waits for a full or an empty buffer has to unlock, lock and look again in a loop (busy waiting), and it uses the processor for nothing. **Condition variables do not fix a bug here, they make the waiting cheap:** the result is the same, but the waiting thread sleeps. You need three things:

a mutex	protects the queue, because both threads change it
not_full	a condition variable: the producer waits on it when the buffer is full; the consumer wakes it up after pop()
not_empty	a condition variable: the consumer waits on it when the buffer is empty; the producer wakes it up after push()

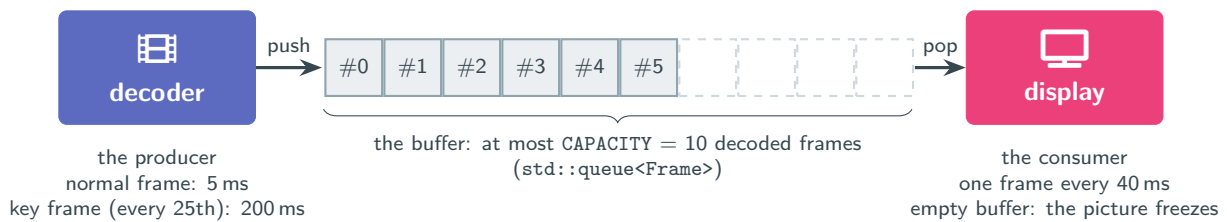
Why a buffer at all? It smooths out the differences in speed. When the producer is slow for a moment, the consumer takes from the stock in the buffer. When the consumer is slow for a moment, the producer puts things into the buffer and does not have to wait.

This pattern is everywhere: a video player, a printer queue, a web server that handles requests, and the pipe in a terminal (`cat file | grep word`: `cat` produces lines, `grep` consumes them).

Producer-consumer: two kinds of waiting (full, empty) around one buffer, protected by one mutex.

The task

You work for a company that writes a driver for video decoding. One thread decodes the frames of a video (the producer), and another one shows them on the screen at a fixed speed: one frame every 40 ms, 25 frames per second (the consumer). Decoding takes an uneven time: a normal frame is fast, a key frame (an I-frame) is much slower. Between the threads there is a buffer of decoded frames. When the buffer is empty, the picture freezes. The driver shows the video correctly, but customers complain that their laptops get hot and the battery runs down fast. Your team must fix it.



Ready code (in video.h, we do not look inside)

Functions

<code>decode_frame(i)</code>	decode frame i: a normal frame takes 5 ms, every 25th frame is a key frame and takes 200 ms
<code>show(f)</code>	show the frame for 40 ms; if it comes more than 10 ms late, print that the picture froze
<code>report()</code>	print how many times the picture froze

Constants

<code>FRAMES</code>	100 frames: the video lasts 4 s
---------------------	---------------------------------

The template is in [Task8](#). This is video.cpp:

```

const int CAPACITY = 10;    // the buffer holds at most 10 frames

std::queue<Frame> buffer;
std::mutex m;

void decoder() {            // the producer
    for (int i = 0; i < FRAMES; i++) {
        Frame f = decode_frame(i);
        m.lock();
        while ((int)buffer.size() == CAPACITY) {    // full? look again
            m.unlock();
            m.lock();
        }
        buffer.push(f);
        m.unlock();
    }
    printf("decoder: all frames decoded\n");
}

void display() {            // the consumer
    while (true) {
        m.lock();
        while (buffer.empty()) {                    // empty? look again
            m.unlock();
            m.lock();
        }
        Frame f = buffer.front();
        buffer.pop();
        m.unlock();
        show(f);
    }
}

int main() {
    std::thread d(decoder), s(display);
    d.join();
    s.join();
    report();
}

```

In the task folder, make run compiles the program and runs it.

Your tasks

- 0 Who is the producer here, who is the consumer, and what is the buffer? When does each of them wait?
- 1 Run it with `time ./build/video`. How much processor time (user) do the threads use? (The program does not end: stop it with Ctrl+C about 2s after decoder: all frames decoded. Question 4 fixes this.)
- 2 Replace busy waiting with two condition variables, `not_empty` and `not_full`. Who waits on which one, and who wakes up whom? Run it with `time` again.
- 3 A colleague put `std::this_thread::sleep_for(std::chrono::milliseconds(10))` into the waiting loops instead. The laptop stays cool. Is this as good a solution as a condition variable?

- 4 The video is over, but the program does not end. Why? Fix it, so that the display shows all frames that are still in the buffer and then ends. (Hint: the decoder must tell the display that nothing more comes.)
- 5 Set `CAPACITY` to 1, then 10, then 100. How many times does the picture freeze? Why does the decoder work ahead? What does a larger buffer cost?

Computer Hardware ■ Lecture 8: Practice with Threads

Task 9: At most 3 decoders at the same time

You have just closed the ticket of Task 8. The next morning you find this e-mail:

✉ Inbox

From: Tomasz M. (Senior Software Engineer, Video Drivers)
To: you
Cc: Video Drivers Team
Subject: RE: [VD-1482] High CPU usage in the display module: semaphores as an alternative

Hi,

first of all congratulations, and thanks for closing VD-1482 so quickly. I did the review: CPU usage at idle dropped practically to zero, and QA confirms that the test laptops no longer get hot. The customer has been informed too. Good job.

While we are at it, I'd like to point you to one more tool that you will meet quite often in our code base: **semaphores**. Here is a short summary, so that we have the same context.

What a semaphore is. It is a counter that can never go below zero. It has two operations:

- `acquire()`: if the counter is greater than zero, decrease it by 1 and return at once. If the counter is 0, the thread sleeps until somebody increases it.
- `release()`: increase the counter by 1 and wake up one of the waiting threads.

Fun fact: semaphores were invented by Dijkstra, the same person who described producer-consumer, and in the same work from 1965 he solved that problem with them.

How to use it in C++. Since C++20 there is `std::counting_semaphore<N>` in the header `<semaphore>`. An example from our own backyard: the GPU has 3 hardware decoding units, so at most 3 streams can be decoded at the same time.

```
std::counting_semaphore<3> hw_decoders{3};    // 3 free units at the start

void decode_stream(int id) {
    hw_decoders.acquire();    // take a unit; if all are taken, sleep
    decode(id);
    hw_decoders.release();    // give the unit back
}
```

How it differs from a mutex. A mutex is in practice a semaphore with a counter of 1, with one important difference: only the thread that locked a mutex may unlock it. `release()` of a semaphore may be called by **any** thread. That is why a semaphore is good not only for limits but also for signalling: one thread calls `release()`, another one waits in `acquire()`.

When a semaphore, and when a condition variable. If you wait for a simple count ("there are free places", "there is something to do"), a semaphore usually gives shorter code. If the condition is more complex, stay with a condition variable.

So I have two requests for this sprint:

1. **[VD-1497]** We have reports that with several videos at once the driver returns a hardware error, because it tries to use a fourth decoding unit, which does not exist. Please make sure that at most 3 streams are decoded at the same time.
2. **[VD-1503]** Try to rewrite the buffer from VD-1482 with two semaphores: one counts the free places, the other one the ready frames. I'd like to compare both versions at the next code review.

If anything is unclear, just write, or catch me at the daily.

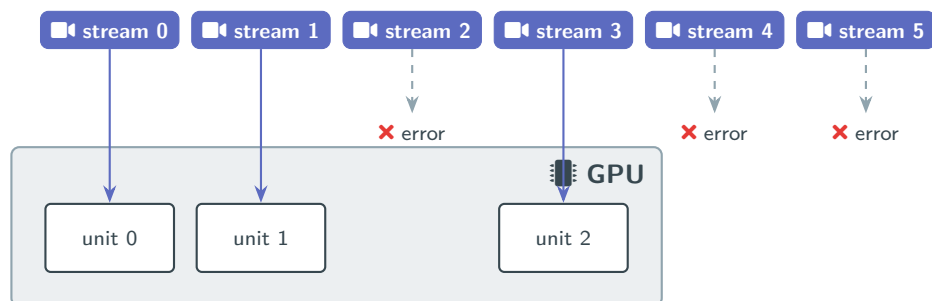
Best regards,
Tomasz

Tomasz M.
Senior Software Engineer ■ Video Drivers Team

This task is ticket VD-1497. Task 10 is ticket VD-1503.

The task: VD-1497

Six videos play at the same time, each one in its own thread. The GPU has only 3 decoding units. The ready function `decode()` checks how many units are in use and reports a hardware error when somebody tries to use a fourth one.



6 streams want to decode at the same time, the GPU has only 3 units

Ready code (in `gpu.h`, we do not look inside)

Functions

<code>decode(id)</code>	decode one stream on a free unit: 1 s; print an error if all 3 units are in use
<code>report()</code>	print how many hardware errors there were

Constants

<code>UNITS</code>	3 decoding units on the GPU
<code>STREAMS</code>	6 videos at the same time

The template is in [Task9](#). This is `streams.cpp`:

```
#include <thread>
#include <vector>
#include "gpu.h"

void decode_stream(int id) {
    decode(id);
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < STREAMS; id++)
        threads.emplace_back(decode_stream, id);
    for (auto &t : threads)
        t.join();
    report();
}
```

In the task folder, make run compiles the program and runs it.

Your tasks

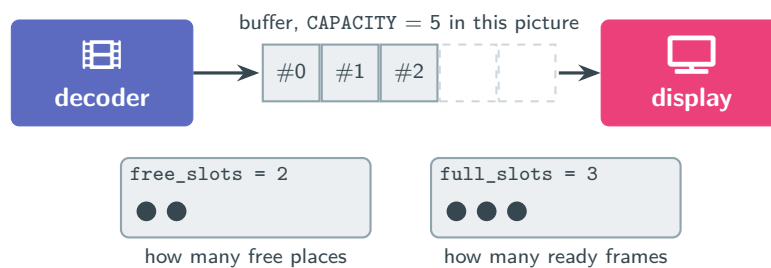
- 1 Run it with `time ./build/streams`. How many streams are decoded at the same time? How many hardware errors are there?
- 2 Add a `std::counting_semaphore` so that at most 3 streams are decoded at the same time. How long does the whole program take now?
- 3 A colleague puts one mutex around `decode()` instead of the semaphore. Do the errors go away? How long does the program take then? Why?
- 4 Somebody forgets `release()` in one place (for example on a path that ends the function early). What happens after a few videos?

Computer Hardware ■ Lecture 8: Practice with Threads

Task 10: The decoder with two semaphores

Ticket VD-1503, from the e-mail of Tomasz in Task 9: “Try to rewrite the buffer from VD-1482 with two semaphores: one counts the free places, the other one the ready frames. I’d like to compare both versions at the next code review.”

The template is the solution of Task 8, with condition variables. Rewrite it with two semaphores:



- `free_slots`: how many free places are in the buffer;
- `full_slots`: how many decoded frames wait in the buffer.

Keep `std::queue`. A semaphore of this size is declared as `std::counting_semaphore<CAPACITY> name{start value};` (header `<semaphore>`).

The code

The template is in [Task10](#). This is `video.cpp`; `video.h` is the same as in Task 8.

```

const int CAPACITY = 10;           // the buffer holds at most 10 frames

std::queue<Frame> buffer;
bool finished = false;           // the decoder has decoded the last frame
std::mutex m;
std::condition_variable not_full; // the decoder waits here when it is full
std::condition_variable not_empty; // the display waits here when it is empty

void decoder() {                  // the producer
    for (int i = 0; i < FRAMES; i++) {
        Frame f = decode_frame(i);
        std::unique_lock<std::mutex> lock(m);
        while ((int)buffer.size() == CAPACITY)
            not_full.wait(lock); // sleep until the display takes a frame
        buffer.push(f);
        not_empty.notify_one(); // the display may be waiting for a frame
    }
    std::lock_guard<std::mutex> lock(m);
    finished = true;             // nothing more comes
    not_empty.notify_one();      // the display may be asleep
    printf("decoder: all frames decoded\n");
}

void display() {                 // the consumer
    while (true) {
        std::unique_lock<std::mutex> lock(m);
        while (buffer.empty() && !finished)
            not_empty.wait(lock); // until a frame comes or the end
        if (buffer.empty()) {    // finished and nothing left: the end
            return;
        }
        Frame f = buffer.front();
        buffer.pop();
        not_full.notify_one();   // the decoder may be waiting for space
        lock.unlock();
        show(f);                 // 40 ms, not under the mutex
    }
}

int main() {
    std::thread d(decoder), s(display);
    d.join();
    s.join();
    report();
}

```

In the task folder, make run compiles the program and runs it.

Your tasks

- ❶ What values do `free_slots` and `full_slots` have at the start? Why?
- ❷ Who calls `acquire()` and who calls `release()` on each of them? Write the new version.
- ❸ With semaphores, do you still need the mutex? What for?
- ❹ A colleague first locks the mutex and only then calls `acquire()` on a semaphore. Write this version and run it. What happens? Why? (Draw who waits for whom.)

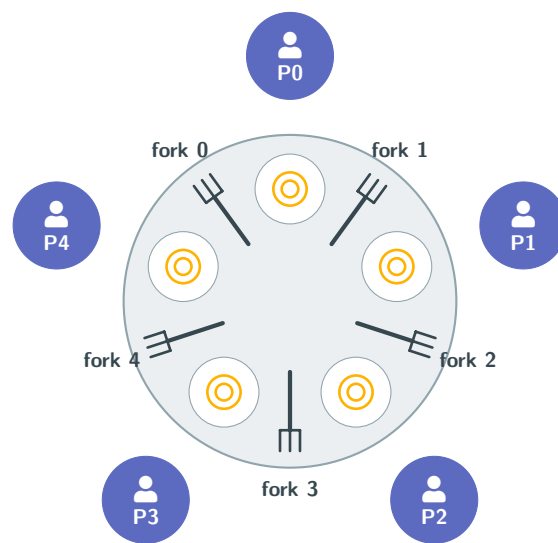
- 5 How do you now tell the display that the video is over? Why does the flag finished from Task 8 not work so easily here?
- 6 Compare your version with the one with condition variables. Which one is shorter? Which one is easier to read? When would you choose a condition variable, and when a semaphore?

Task 11: The dining philosophers

Task 11 is about one of the most famous problems of concurrent programming. Read this introduction first.

The dining philosophers problem

In 1965 Edsger Dijkstra (the same person who described producer-consumer and invented semaphores) gave his students an exam problem about five computers that share five tape drives. A few years later Tony Hoare told the same problem as a story, and this story is how everybody knows it today.



Five philosophers sit at a round table. In front of each of them is a plate of spaghetti, and between every two neighbours lies **one fork**: five forks for five people. A philosopher does only two things, again and again: **thinks** and **eats**. To eat, a philosopher needs **both** forks: the one on the left and the one on the right. A fork can be held by only one philosopher at a time. After eating, the philosopher puts both forks back and thinks again.

Why is this important? The philosophers are threads, and the forks are resources that two threads share: locks, files, devices, database rows. Every program in which a thread needs **two or more** shared resources at the same time has the same problem. The table only makes it easy to see.

What can go wrong? The simplest rule is: “take the left fork, then the right fork”. Imagine that all five philosophers get hungry at the same moment, and each one takes the left fork. Now every fork is in someone’s hand, and every philosopher waits for the right fork, which the neighbour holds and will never put down. Nobody eats, ever. This is a **deadlock**, and you know its shape from Tasks 4 and 5: a **circle of waiting**, here all the way around the table.

There is a second danger: **starvation**. Even without a deadlock, a rule could be unfair, so that one philosopher almost never gets both forks while the neighbours eat again and again. A good solution has neither a deadlock nor starvation.

Deadlock and starvation

Deadlock: a group of threads waits in a circle, and none of them can ever go on.

Starvation: the program goes on, but one thread waits much longer than the others, or for ever.

The program

Every philosopher is a thread, and every fork is a `std::mutex`. Philosopher `i` has fork `i` on the left and fork `(i + 1) % 5` on the right. Everybody takes the left fork first. The dinner lasts 3s; then the program prints how many times each philosopher has eaten.

Ready code (in `table.h`, we do not look inside)**Functions**

<code>think(id)</code>	think: 0 to 9 ms
<code>pick_up(fork)</code>	pick up a fork: lock its mutex
<code>eat(id)</code>	eat: 5 ms, and count the meal
<code>wait_for_end()</code>	wait 3s, then end the dinner
<code>report()</code>	print the number of meals of every philosopher

Constants and variables

<code>N</code>	5 philosophers and 5 forks
<code>dinner_over</code>	becomes true when the dinner ends

The template is in [Task11](#). This is `philosophers.cpp`:

```
std::mutex forks[N];           // fork i: between philosopher i - 1 and philosopher i

void philosopher(int id) {
    int left = id;             // the fork on my left
    int right = (id + 1) % N;  // the fork on my right
    while (!dinner_over) {
        think(id);
        pick_up(forks[left]);
        pick_up(forks[right]);
        eat(id);
        forks[right].unlock();
        forks[left].unlock();
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < N; id++)
        threads.emplace_back(philosopher, id);
    wait_for_end();
    for (auto &t : threads)
        t.join();
    report();
}
```

In the task folder, `make run` compiles the program and runs it.

Your tasks

- ❶ Run the program several times. Does it always end? (A program that hangs: stop it with Ctrl+C.)
- ❷ When it hangs, draw the table: which fork does each philosopher hold, and which one does each wait for? Where is the circle?
- ❸ **Fix 1:** one philosopher takes the forks in the other order. Rule: everybody takes the fork with the **smaller number** first. Which philosopher changes? Why does this break the circle?
- ❹ **Fix 2:** go back to the original code. A waiter lets at most **4** philosophers sit at the table at the same time; the fifth one waits until somebody stands up. Write it with a `std::counting_semaphore` (Task 9). Why are 4 enough to prevent the deadlock?
- ❺ Look at the numbers of meals in both fixes. Does anybody starve? Is one fix fairer than the other?